

|                   |               |                     |                     |
|-------------------|---------------|---------------------|---------------------|
| CANDIDAT<br>_____ | DATE<br>_____ | DURÉE<br>10 minutes | NOTE<br><b>/ 20</b> |
|-------------------|---------------|---------------------|---------------------|

## LA CONSIGNE

Cochez vos réponses, puis comptez vos points : **le corrigé est en dernière page**. 5 questions, 4 points chacune.

5 questions

une seule bonne réponse

4 points par bonne réponse

★ = piège classique

## I WINDOW FUNCTIONS SQL

questions 1 à 5 ●●●●●

**Q1** Que produit `ROW_NUMBER() OVER(ORDER BY x)` sur un ensemble de lignes ? ★

- ☐ Un numéro de ligne unique et séquentiel (1..N), sans ex æquo.
- ☐ Un rang partagé en cas d'égalité, avec des trous dans la séquence.
- ☐ Un rang partagé en cas d'égalité, sans trous dans la séquence.
- ☐ Une somme cumulée des valeurs de la colonne x.

**Q2** Quelle est la différence entre `RANK()` et `DENSE_RANK()` en cas d'ex æquo ? ★

- ☐ `RANK` est plus performant ; `DENSE_RANK` est déprécié.
- ☐ `RANK` laisse des trous dans la séquence (1,1,3) ; `DENSE_RANK` n'en laisse pas (1,1,2).
- ☐ `DENSE_RANK` laisse des trous ; `RANK` n'en laisse pas.
- ☐ Les deux fonctions sont strictement équivalentes.

**Q3** Que retourne `LAG(ventes, 1) OVER(ORDER BY date)` pour une ligne donnée ?

- ☐ La valeur de la ligne suivante dans l'ordre défini.
- ☐ La somme cumulée depuis la première ligne jusqu'à la ligne courante.
- ☐ La valeur de la ligne précédente dans l'ordre défini.
- ☐ La valeur minimale de la partition.

**Q4** Quelle syntaxe SQL calcule un **running total** (somme cumulée du début à la ligne courante) ? ★

- ☐ `SUM(x) OVER(ORDER BY date)`
- ☐ `SUM(x) OVER(ORDER BY date ROWS UNBOUNDED PRECEDING)`
- ☐ `SUM(x) OVER(PARTITION BY date)`
- ☐ `CUMSUM(x) OVER(ORDER BY date)`

**Q5** Que se passe-t-il si on utilise `OVER()` sans clause `ORDER BY` ?

- ☐ La requête génère une erreur de syntaxe.
- ☐ La fenêtre est vide : la fonction retourne NULL.
- ☐ L'agrégat porte sur toute la partition (total du groupe), pas un cumul ordonné.
- ☐ Le moteur applique automatiquement un `ORDER BY` sur la clé primaire.

- ✓ **Q1 : Numéro de ligne unique et séquentiel (1..N), sans ex æquo.** ROW\_NUMBER attribue un entier distinct à chaque ligne, même si deux lignes ont la même valeur de x. Contrairement à RANK ou DENSE\_RANK, il ne partage jamais le même rang.
- ✓ **Q2 : RANK laisse des trous (1,1,3) ; DENSE\_RANK n'en laisse pas (1,1,2).** Sur deux lignes ex æquo au rang 1, RANK saute le rang 2 et passe directement au rang 3 ; DENSE\_RANK continue avec 2. Le choix dépend de la sémantique métier voulue.
- ✓ **Q3 : La valeur de la ligne précédente.** LAG(col, n) remonte de n lignes en arrière dans l'ordre défini. Son symétrique est LEAD(col, n) qui avance de n lignes. Utile pour calculer des variations : ventes - LAG(ventes, 1).
- ✓ **Q4 : SUM(x) OVER(ORDER BY date ROWS UNBOUNDED PRECEDING).** La clause ROWS UNBOUNDED PRECEDING fixe le début de la fenêtre à la première ligne de la partition et la fin à la ligne courante. Sans elle, le moteur peut appliquer une fenêtre par défaut différente selon le dialecte SQL.

- ✓ **Q5 : L'agrégat porte sur toute la partition, pas un cumul ordonné.** OVER ( ) sans ORDER BY définit une fenêtre couvrant toutes les lignes de la partition. Utile pour calculer la part du total : SUM(x) OVER(PARTITION BY cat) donne le total par catégorie sur chaque ligne, sans cumul.

**VOTRE NOTE SUR 20****0-8**

Les window functions méritent une révision complète · relire la doc de votre moteur SQL.

**12-16**

Profil opérationnel. Maîtrisez les clauses ROWS/RANGE et les pièges de fenêtre par défaut.

**20**

Score parfait · ce test ne devrait plus vous arrêter en entretien ou en code review.

· **Postez votre note en commentaire, sans tricher.**